

Operation TELESCREEN - Requirements & Grading Criteria

LEGAL DISCLAIMER: The information, tools, and code provided in this repository and course are strictly for educational, research, and defensive purposes only.

You are explicitly prohibited from using any materials contained herein to access, test, modify, or exploit any device, network, or system that you do not own 100% or for which you do not have explicit, documented, and legally binding authorization to interact with.

By using this repository and course, you acknowledge and agree that:

1. Any illegal, unauthorized, or malicious use of this information is solely your responsibility.
2. The author(s) and contributor(s) of this repository and course shall not be held liable for any damages, legal repercussions, criminal charges, or unauthorized actions resulting from the use, misuse, or abuse of the contents herein.
3. You will comply with all applicable local, state, national, and international laws regarding cybersecurity and computer fraud.

IF YOU DO NOT AGREE WITH THESE TERMS, DO NOT USE THIS REPOSITORY AND COURSE.

Project Overview

You are an embedded reverse engineer in the Brotherhood signals lab. The Party has deployed four million TELESCREEN units - combined surveillance cameras and wireless routers - each running the **same four-partition firmware** (boot, bootargs, kernel, rootfs). Your mission is to reverse engineer the captured image, expose every backdoor, break the exfiltration cryptography, and build a **Raspberry Pi 5 replica** that boots the *same four-partition layout* with a hardened **AES-256-GCM** channel.

This is an application-class project. The target is a real Linux system on a Cortex-A SoC (the TELESCREEN is a BCM2712; the replica is a BCM2712). There is **no bare metal**, no Pico, no .uf2. The four partitions, the U-Boot environment, the vendor kernel container, and the JFFS2 rootfs are **identical in format** to the TELESCREEN.

This project covers the full embedded-security arc: flash layout and partitioning, ARM first-stage boot, U-Boot and the CRC-protected environment, the vendor kernel container, the device tree, JFFS2 nodes and `crc32_le`, the userland boot chain, the HTTP dispatcher and `system()` injection, backdoor classes, weak key derivation versus AEAD, building an RP5 router, and blue-team detection.

Learning Objectives

Upon completion of this project, you will demonstrate the ability to:

1. Carve a whole-flash image into the four partitions and identify each by **magic bytes**
 2. Trace an **ARM application-class boot chain** from the first stage to the application
 3. Read a **CRC-protected U-Boot environment** and decode `bootargs/bootcmd`
 4. Parse the **vendor kernel container** and inflate it to a raw ARM Image
 5. Extract and read a **device tree blob** and the kernel command line
 6. Read and write **JFFS2** using `crc32_le` with correct CRC discipline
 7. Reverse a stripped **ARM Cortex-A application** in Ghidra and locate the dispatcher and `system()` sites
 8. Explain why **key derivation from public identifiers is not encryption**, and contrast it with **AEAD**
 9. Build an **RP5 router** (Wi-Fi AP + Ethernet WAN, NAT, DHCP/DNS)
 10. Reproduce the **four-partition layout on RP5** and boot a real Linux kernel from it
 11. Detect and defend against **beaconing, backdoors, and weak key derivation**
 12. Analyse the **ethical and legal dimensions** of surveillance-device research
-

Deliverables Checklist

You must submit **all** of the following. Missing deliverables will result in zero points for the corresponding task.

#	Deliverable	Format	Task
1	Partition table (offset/size/name) + magic-byte identification for all four partitions + the decoded <code>mtddparts</code> string	Inside TELESCREEN– <code>Answers.md</code>	Task 1
2	Boot-chain analysis: first-stage entry point, cold-boot call addresses, U-Boot version/build string, decoded <code>bootargs/</code> <code>bootcmd</code>	Inside TELESCREEN– <code>Answers.md</code>	Task 2
3	Kernel container header table, raw Image size, device-tree machine model + flash node, kernel banner	Inside TELESCREEN– <code>Answers.md</code>	Task 3
4			Task 4

#	Deliverable	Format	Task
	Rootfs tree summary, the two CRC algorithms compared, and your in-place JFFS2 patch with before/after CRC values	Inside TELESCREEN-Answers.md	
5	Backdoor catalogue B1-B5 : address/offset, code or script, one-line exploit path each	Inside TELESCREEN-Answers.md	Task 5
6	Crypto break: the key-derivation formula, a working decryptor, and a 200-word analysis	Inside TELESCREEN-Answers.md	Task 6
7	RP5 lab: four-partition image layout, U-Boot configuration, router proof (a client getting DHCP through the AP), and the AEAD beacon proof	Inside TELESCREEN-Answers.md	Task 7
8	Defensive playbook (bonus)	Inside TELESCREEN-Answers.md	Task 8
9	Full submission packaged as lastname-firstname-TELESCREEN.zip	ZIP	-

Required Tools and Equipment

Tool	Purpose	Required For
A flash dump of the TELESCREEN (or the instructor-issued full.img)	The artifact under analysis	Tasks 1-6
dd, binwalk, xxd, strings	Carving and identification	Task 1
arm-none-eabi-objdump / Ghidra	Disassembly and analysis	Tasks 2, 5, 6
jefferson (or a JFFS2 parser)	Rootfs extraction	Task 4
Python 3	Tooling, the decryptor, verification	Tasks 1, 4, 6
Raspberry Pi 5 + microSD/ NVMe + U-Boot + a Linux build host	The teaching replica	Task 7

Tool	Purpose	Required For
USB-UART adapter	Boot console (115200 8N1)	Tasks 2, 7
Wi-Fi client device	Prove the AP hands out DHCP	Task 7

Artifact Identity

The instructor-issued artifact hashes are:

```
TELESCREEN-full.img
037cc979a58f04f3285132a7b745ca859b1604aeb80c9ef62eb5377e48bd18ec
TELESCREEN-boot.img
4193d6a9e9a29848813056f05356c17e4a4e64f8206148c86fd3b0d50a5e69fd
TELESCREEN-env.img
e935f998fd6289615d88d8d2c2d4d3aada0e04ad6dd7592c27a853431fec4d7e
TELESCREEN-kernel.img
8a5fb5725420d79ae4e21e39e27c161b7df88dbbdbc2b061d859dd889c8b0230
TELESCREEN-rootfs.img
38a8684afb5f8bb8b717b2ab333dc729054d36192aa6541be6e9fed6f19cd10e
```

The real hashes are published with the instructor-issued image. Verify them before you begin; a mismatch means you are analysing the wrong artifact.

Grading Rubric - Detailed Breakdown

Task 1: Carve and Identify the Four Partitions (10 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Whole-image hash verified	2	In Answers.md, the hash matches the instructor-issued value	Hash computed but not compared	Not done
[DOCUMENT] Partition table correct	3	In Answers.md, all four partitions with correct offset and size, and the decoded mtdparts string	Offsets correct, sizes missing	Not found
	3	In Answers.md,	2-3 magics correct	0-1 correct

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Magic-byte identification		the correct magic for each partition (boot vectors, env CRC+text, kernel 21 84 1b 00, rootfs 85 19 03 20)		
[DOCUMENT] Carved files produced	2	Four carved files exist and are the correct sizes	Some files produced	None

Task 2: Reverse the Boot Chain (15 points)

Objective: Trace the device from power-on to the kernel hand-off.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] First-stage entry point identified	4	In Answers.md, the reset-vector entry address documented (vector table at offset 0)	Entry region noted, address wrong	Not found
[DOCUMENT] Cold-boot path traced	4	In Answers.md, at least two DRAM/SoC initialisation call addresses documented	One call documented	None
[DOCUMENT] U-Boot version/build string	3	In Answers.md, the exact version and build string recorded	Version only	Not found
[DOCUMENT] bootargs/bootcmd decoded	4	In Answers.md, both decoded, including the mtdparts string and the kernel load command	One of the two decoded	Neither

Task 3: Inflate the Kernel and Read the Device Tree (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Container header parsed	5	In Answers.md, the magic, the length field, and the gziphead tag documented	Some fields	Not found
[DOCUMENT] Raw Image produced	5	In Answers.md, the inflated size and first bytes recorded; file produced	Inflated but size wrong	Not inflated
[DOCUMENT] Device tree + banner	5	In Answers.md, the machine model, the flash-controller node, and the kernel version/ build string	Partial	Not found

Task 4: Open the Rootfs (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Rootfs extracted	4	In Answers.md, the application, web directory, and config directory listed	Partial tree	Not extracted
[DOCUMENT] CRC algorithms compared	6	In Answers.md, the U-Boot env CRC32 and the JFFS2 crc32_le both named and contrasted	One named	Neither
[DOCUMENT & PATCH] In- place patch verified	5	In Answers.md, the patch with before/after hdr_crc/ data_crc/	Patch done, CRCs not shown	No patch

Criterion	Points	Full Credit	Partial Credit	No Credit
		node_crc values		

Task 5: Find the Backdoors (20 points)

Objective: Locate and explain the exploit path for B1-B5.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] B1 - config-sourced root exec	4	In Answers.md, the init path and the sourced config file, with the exploit path	Path found, no exploit line	Not found
[DOCUMENT] B2 - CGI dispatcher command injection	4	In Answers.md, the dispatcher address and at least one system() site it reaches	Dispatcher or site only	Not found
[DOCUMENT] B3 - archive-to-root restore	4	In Answers.md, the tar ... -C / site and the proof it is network-reachable	Site only	Not found
[DOCUMENT] B4 - credential store	4	In Answers.md, the credential file and the empty/default login demonstrated	File only	Not found
[DOCUMENT] B5 - debug shell	4	In Answers.md, the shell and its trigger documented	Shell only	Not found

Task 6: Break the Exfiltration Crypto (15 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
	6			Not found

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Key-derivation schedule recovered		In Answers.md, the exact formula (e.g. <code>key = f(MD5(id1 & id2 * id3))</code>)	Method named, formula wrong	
[PATCH] Decryptor works	5	Grader runs your decryptor on a captured beacon and obtains readable telemetry	Decryptor runs but output wrong	Not submitted
[DOCUMENT] 200-word analysis	4	Articulates why a key derived from a public identifier provides no confidentiality	Partial	Not addressed

Task 7: Build the RP5 TELESCREEN-Lab (10 points)

Criterion	Points	Full Credit	Partial Credit	No Credit
[PATCH] Four-partition layout reproduced	3	Grader confirms the RP5 image carves into <code>boot/</code> / <code>bootargs/</code> / <code>kernel/</code> / <code>rootfs</code> with the same offsets and sizes	Layout present, offsets wrong	Not done
[PATCH] RP5 boots the image	3	Serial console shows U-Boot -> Linux -> the application	Boots to U-Boot only	Does not boot
[DOCUMENT] Router proof	2	A Wi-Fi client obtains DHCP through the AP and reaches the WAN	AP up, no DHCP	Not done
[DOCUMENT] AEAD proof	2	The beacon is now AES-256-GCM; a forged frame is	AEAD used, no forgery test	Not done

Criterion	Points	Full Credit	Partial Credit	No Credit
		rejected at the tag		

Task 8: The Defensive Playbook (bonus, 10 points)

Requirement: A one-page blue-team playbook covering detection of the beacon, the backdoors, and the weak key derivation, plus field hardening.

Criterion	Points	Full Credit	Partial Credit	No Credit
[DOCUMENT] Beacon detection is specific	4	Names periodicity/jitter analysis and payload-entropy analysis with a concrete method	Generic “watch the network”	Not addressed
[DOCUMENT] Backdoor detection is specific	3	Names at least two concrete indicators (e.g. the sourced config, the restore endpoint)	One indicator	Not addressed
[DOCUMENT] Field hardening is practical	3	Names specific measures (secure boot, signed images, key management) with justification	Generic	Not addressed

Written Analyses (Q1-Q3)

Criterion	Points	Full Credit	Partial Credit	No Credit
Q1 The tradecraft parallel (300 words)	5	Draws specific parallels between this device and real-world backdoored cameras/routers, and explains why “sealed” channels fail	Generic comparison	Not addressed

Criterion	Points	Full Credit	Partial Credit	No Credit
Q2 Ethics and law (300 words)	5	Balanced analysis; discusses authorisation, proportionality, and disclosure	One-sided	Not addressed
Q3 Defensive recommendations (3 proposals)	5	One hardware, one firmware, one operational measure, each specific	1-2 specific	Not addressed

Common Pitfalls

These are the most frequent mistakes. Avoid them.

Pitfall	Consequence	How to Avoid
Treating the kernel as a zImage/uImage	You will not find 0x016F2818 or 0x27051956; you will think the image is corrupt	Parse the vendor container (21 84 1b 00 + "gziphead" + gzip "Image")
Using zlib's CRC for JFFS2	Your patched node fails to mount; the file is ignored	JFFS2 uses crc32_le (reflected, no init/final inversion)
Editing the U-Boot environment without fixing the CRC	U-Boot falls back to built-in defaults; your edit is ignored	Recompute the leading CRC32(LE) over the env blob
Looking for .cgi files on the filesystem	Only two are real files; the rest are internal routes	Treat the HTTP dispatcher as code, not files
Searching for system() calls in the wrong binary	You will miss the injection sites	Follow the dispatcher to the application's system() sites
Assuming "AES" means "secure"	The Ministry's key is derived from a public ID	Ask where the key comes from , not which cipher is used
Reusing a GCM nonce in the lab	Catastrophic forgery; you have taught the wrong lesson	One nonce per message, never reused
Changing JFFS2 node length	All subsequent nodes shift; the filesystem is corrupt	Replace with the same total length (INODE + PADDING)

How To Bench the RP5 Replica

Wire and configure the replica exactly as follows:

- **Image store (microSD or NVMe):**
 - Lay out four partitions at the locked offsets: boot (128 KiB), bootargs (64 KiB), kernel (1792 KiB), rootfs (14400 KiB). Leave the vendor pad before rootfs.
- **WAN:**
 - Ethernet (eth0) to the lab uplink (or the local collector).
- **LAN / AP:**
 - On-board Wi-Fi (wlan0) in AP mode via hostapd.
- **Console:**
 - USB-UART on the debug header, **115200 8N1**, to watch U-Boot -> Linux -> app.

CRITICAL REMINDER: Keep the lab **isolated**. The collector (lab-sink) is **local**. Do not bridge the replica's WAN to any production network.

Partition / Memory Reference

Partition	Offset	Size	Magic / identifier
boot	0x000000	128 KiB	ARM vectors at offset 0; U-Boot string inside
bootargs	0x020000	64 KiB	CRC32 (LE) then key=value\0 text
kernel	0x030000	1792 KiB	21 84 1b 00 + gziphead + gzip Image
rootfs	0x1F0000	14400 KiB	85 19 03 20 (JFFS2 little-endian)

Key Constants Reference

Constant	Value	Purpose
MTD_BOOT_OFF	0x000000	boot partition offset
MTD_BOOTARGS_OFF	0x020000	environment partition offset
MTD_KERNEL_OFF	0x030000	kernel partition offset
MTD_ROOTFS_OFF	0x1F0000	rootfs partition offset
CONTAINER_MAGIC	0x001B8421	vendor kernel container
JFFS2_MAGIC	0x1985	JFFS2 little-endian magic
JFFS2_CLEANMARKER	0x2003	first node type
JFFS2_ERASEBLK	0x10000	64 KiB erase block

Deadline & Submission

- This is a **take-home final project**. See the course syllabus for the due date.
 - **Submission Format:** create a folder holding all deliverables (TELESCREEN-Answers.md, the Ghidra screenshot(s), the carved images, your decryptor, and the RP5 image layout).
 - ZIP this folder and name it: lastname-firstname-TELESCREEN.zip.
 - **Example:** if your name is Kevin Thomas, your file must be exactly thomas-kevin-TELESCREEN.zip.
 - Late submissions: see the syllabus late-work policy.
-

Grade Scale

Grade	Percentage	Points
A+	97-100%	97-100
A	93-96%	93-96
A-	90-92%	90-92
B+	87-89%	87-89
B	84-86%	84-86
B-	80-83%	80-83
C	70-79%	70-79
F	0-70%	0-70

Partial Credit Policy

Scenario	Credit
Correct concept and approach, wrong specific offset or byte value	75% of task points
Identified the partition/location correctly but the patch is incorrect	60% of task points
Explained the concept correctly but could not locate it in the image	40% of task points
Patched correctly but could not explain why the fix works	50% of task points
Documented the analysis process thoroughly even though the result is wrong	30% of task points
Three partitions carved, one missing	70% of Task 1 points
RP5 boots but the router does not hand out DHCP	50% of Task 7 points
Beacon uses AEAD but nonce discipline is not demonstrated	40% of the AEAD criterion

Academic Integrity

By submitting this final project, you certify that: 1. This is your own work 2. You have not shared answers with other students 3. You understand the ethical implications of embedded security research 4. You understand that the skills demonstrated here may only be used in authorized, lawful contexts 5. You recognize the difference between academic exercises and real-world operations

Reference Material

- ARM Cortex-A76 Technical Reference Manual
- ARMv8-A Cryptography Extensions (AES/SHA/PMULL)
- Ghidra documentation: <https://ghidra-sre.org/>
- JFFS2 format (Linux `mtd-utils` / `jffs2dump`)
- U-Boot environment and `mtdparts` documentation
- NIST SP 800-38D (GCM) and RFC 8439 (ChaCha20-Poly1305)
- OpenIPC / Xiongmai camera-firmware research (prior art)